

Gedächtnisprotokoll :: Prüfung Algorithmen und Datenstrukturen 1 (Informatik)

Max Fischer, 1. Februar 2019

Aufgabe 1 [8 Punkte]

Multiple Choice. Eine, mehrere oder keine Antworten können richtig sein.

Falsch Angekreuztes gibt einen Punkt Abzug.

Falls die ganze Aufgabe 1 negativ wird, wird sie mit 0 Punkten bewertet.

a) Wovon hängt die Suchzeit im Trie ab? Die Kindzeiger sind in einer geordneten Liste gespeichert.

- ☐ Größe des Alphabets (k)
- ☐ Länge des gesuchten Wortes (l)
- ☐ Anzahl der Einträge (n)

b) Wovon hängt die Speicherkomplexität im Trie ab?

Größe des Alphabets (k), Anzahl der Einträge(n), Maximallänge der Schlüsselwörter (l).

- ☐ hängt von k ab
- ☐ proportional zu k
- ☐ hängt von l ab
- ☐ proportional zu l
- ☐ hängt von n ab
- ☐ proportional zu n

c) Wofür eignet sich die Trie-Struktur gut?

- ☐ Präfix-Suche (gegebener Wortanfang und die Wortvervollständigungen suchen)
- ☐ Postfix-Suche (gegebenes Wortende und die Wortvervollständigungen suchen)
- ☐ exakte Suche (gegebenes Wort, Suche ob es vorhanden ist)

d) Wo ist Fädelung sinnvoll?

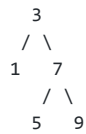
- ☐ hohler Baum (Daten nur in Blättern)
- ☐ Binärbaum (Daten in Blättern & Knoten)
- ☐ Doppelt verkettete Liste

e) Was gilt für eine statische Hashtabelle?

- ☐ Ist auf die Länge des zugrundeliegenden Arrays beschränkt.
- ☐ Die Zeitkomplexität der Suche ist im Best-Case $O(\log n)$.
- ☐ Kann manchmal einen Belegungsfaktor $\alpha > 1$ haben
- ☐ Suche ist immer schneller als bei einem AVL-Baum mit derselben Anzahl an Elementen.

Aufgabe 2 [7 Punkte]

Gegeben sei folgender AVL-Baum:



a) Zwei Operationen werden durchgeführt:

- Es wird in obigen Baum eine 8 eingefügt.
- Es wird in obigem Baum (ohne 8) die 1 gelöscht.

Zeichnen Sie jeweils den resultierenden Baum.

b) Wie heißt die für das AVL-Kriterium wichtige Kennzahl?

c) Notieren Sie die Kennzahl an den Knoten in den obigen Bäumen.

d) Welche Vorteile und Nachteile haben AVL-Bäume gegenüber "normalen" Binärbäumen?

e) Welche Komplexität haben die für AVL-Bäume relevanten Operationen?

Aufgabe 3 [8 Punkte]

Gesucht ist die Zeitkomplexität (in O-Notation) im **ungünstigsten** Fall für die folgenden Operationen

- Suchen
- Einfügen
- Löschen
- Erstellen (durch iteriertes Einfügen, die Anzahl der Elemente sei zu Anfang bekannt)

in den Datenstrukturen

- ungeordneter Vektor
- geordnete verkettete Liste
- natürlicher binärer Suchbaum
- offenes Hash-Verfahren[sic!]

Denken Sie an Degenerierung!

Aufgabe 4

Modifizieren Sie die gegebenen Queue-Axiome so, dass sie für eine **Prioritätswarteschlange** zutreffen.

Operationen:

```
new : → Queue<T>
empty: Queue<T> → bool
push: Queue<T> × T → Queue<T>
pop: Queue<T> → Queue<T>
top: Queue<T> → T
```

Vorbedingungen:

```
pre(pop(t)) := !empty(t)
pre(front(t)) := !empty(t)
```

Axiome:

```
empty(new())
!empty(push(q,t))
pop(push(q,t)) = empty(q) ? q : push(pop(q),t)
top(push(q,t)) = empty(q) ? t : top(q)
```

Schreiben Sie das angepasste Axiom in den jeweiligen Kasten.

Aufgabe 5

- a) Was ist die untere Grenze für die Zeitkomplexität eines **allgemeinen** Sortierverfahrens für Arrays der Länge n ? Verwenden Sie nur Ω , O , Θ Notation.
- b) Nennen Sie ein Sortierverfahren, welches diese Grenze selbst im ungünstigsten Fall erreicht.
- c) Welche Zeitkomplexität und Speicherkomplexität hat ein (gut implementierter) QuickSort im ungünstigsten Fall?

Aufgabe 6 [10+ Punkte]

C++ Code schreiben: Die Mengenoperation (Vereinigung, Schnitt, Element von) sind in C++ nicht auf Containern selbst implementiert, sondern generisch anhand von Iteratoren.

- a) Schreiben sie eine Funktion `intersect`, die die Schnittmenge zweier *sortierter* Container (z. B. `set`) berechnet und in einen neuen Container schreibt.

```
template <class In, class Out>
Out intersect(In first1, In last1, In first2, In last2, Out result) {
    ...
}
```

- b) Geben Sie die Zeitkomplexität Ihrer Implementierung in Abhängigkeit der Längen

- $n1$: $|[first1, last1)|$
- $n2$: $|[first2, last2)|$

für den günstigsten und ungünstigsten Fall an.